CODEFORCES · C++ · MACOS AND LINUX

Set up a contest-ready C++ workspace in one command.

Problems import themselves from your browser. Your editor runs the samples with a key press. Random stress tests check your solution against a brute force. Works with VS Code + Vim and Sublime Text + Vim.

terminal

```
$ curl -fsSL https://raw.githubusercontent.com/bobikenobi12/cf-kit/main/install.sh | bash
```

Install command	Editors supported	Snippets included	Random tests per stress run
1	2	8	500

QUICKSTART

Four steps, about five minutes.

You need git, Python 3.9+, and GCC. The installer checks for GCC and offers to install it with Homebrew on macOS.

1. **Run the install command** above. It downloads cf-kit to `~/cf-kit` and asks a few yes/no questions. Press Enter to accept the defaults.
2. **Install the browser extension** *Competitive Companion*. The installer links to it.
3. **Start the listener** and leave it running: `python3 ~/cf-kit/scripts/listen.py` (or `Space` `l` in VS Code).
4. **Click the extension** on any Codeforces problem. Its folder appears in `contests/`, with the samples and a solution file ready.

> **Want to try it first?** Run `python3 scripts/judge.py examples/sum-of-two/sol.cpp`. The example ships with the repository and passes its samples.

Options for the installer

COMMAND	WHAT IT DOES
`... | bash -s -- --yes`	Accept every default without asking. Good for scripts.
`... | bash -s -- --skip-editors`	Don't change VS Code or Sublime Text. Only the compiler and header.
`python3 scripts/doctor.py`	Check the setup again at any time. Prints what is wrong and how to fix it.

HOW IT WORKS

From a problem to an accepted solution.

1. Import

automatic

Competitive Companion sends the statement's samples to a local listener on port 27121. Each problem gets its own folder, named from its URL.

2. Write

your code

`sol.cpp` opens from a template with fast I/O, short aliases, and a debug macro that disappears on Codeforces.

3. Check

one key

The judge compiles with the same flags as the release build and runs every sample with its time limit. It shows the first difference when an answer is wrong.

4. Stress

before submitting

Your solution is compared with a slow brute force on random inputs. The first mismatch is saved so you can reproduce it.

Folder layout

```
cf-kit/
├── install.sh              one-line bootstrap
├── setup/install.py        interactive installer
├── scripts/
│   ├── listen.py           receives problems from the browser
│   ├── judge.py            runs samples; --debug adds sanitizers
│   ├── stress.py           random testing against brute.cpp
│   ├── doctor.py           checks everything, says how to fix it
│   ├── common.py           compiler, flags, comparison
│   └── pch.py              builds the precompiled header
├── templates/              sol.cpp, brute.cpp, gen.py
├── examples/sum-of-two/    a problem to try the tools on
├── .vscode/                settings, tasks, snippets
└── contests/               your problems (kept out of git)
```

EDITORS

Vim keys, in both editors.

VS Code + Vim

The installer configures the Vim extension and C/C++ IntelliSense. Leader is `Space`.

KEYS	ACTION
`Space` `b`	Judge the current file on the samples
`Space` `d`	Judge with sanitizers and bounds checks
`Space` `s`	Stress test this problem
`Space` `l`	Start the problem listener
`j` `k`	Leave insert mode
`Cmd` `Shift` `B`	Judge (default build task, works without Vim)

Sublime Text + Vim

The installer enables Sublime's built-in Vintage mode and adds a build system.

- **Judge:** `Cmd` `B` with *Tools > Build System > CF-C++* selected. Select it once per window.
- **Debug build:** *Tools > Build With > Debug*.
- **Errors:** the output panel links jump to the line.
- **Snippets:** type the prefix, then `Tab`.

THE TEMPLATE

Fast I/O and minimal boilerplate.

Every new `sol.cpp` starts from this file. The lines that matter for speed and correctness:

```
ios::sync_with_stdio(false);   // C++ streams stop syncing with C stdio: much
faster I/O
cin.tie(nullptr);              // cout is no longer flushed before each cin read

using ll = long long;          // use ll whenever values can exceed 2*10^9
#define all(x) begin(x), end(x)

#ifdef LOCAL
#define dbg(x) cerr << "[" << __LINE__ << "] " << #x << " = " << (x) << '\n'
#else
#define dbg(x) ((void)0)           // compiles to nothing on Codeforces
#endif
```

- **`'\n'`, not `endl`.** `endl` flushes on every line, which is slow for large output.
- **`ll` instead of `long`.** Codeforces compiles on Windows, where `long` is 32 bits. On macOS and Linux it is 64 bits, so code that relies on it can pass locally and fail on the judge.
- **Debug output is free.** `dbg(x)` prints to stderr only when `LOCAL` is defined, which the local tools do. You can leave it in submitted code.

SNIPPETS

Eight snippets, same in both editors.

PREFIX	EXPANDS TO
`cp`	The whole template
`forn`	`for (int i = 0; i < n; ++i)`
`forr`	Loop from `n-1` down to `0`
`vin`	Read `n` values into a vector
`bs`	Binary search for the smallest `x` where `ok(x)` holds
`mpow`	Modular power by repeated squaring
`yn`	Print `YES` or `NO`
`dbg`	Debug print

Snippets live in `.vscode/cpp.code-snippets`. The installer converts them for Sublime, so edit one file and run the installer again.

TESTING

Three levels, from fast to thorough.

TOOL	WHAT IT CHECKS	WHEN
`judge.py`	Your solution on the samples. Reports OK, WA with a diff, RE, or TLE.	After every change
`judge.py --debug`	Same samples, with AddressSanitizer, UBSan and bounds-checked STL. Stops at the first bad access.	When a sample fails oddly, before submitting
`stress.py`	Your solution against `brute.cpp` on 500 random inputs from `gen.py`.	When the samples pass but you're not sure

Why the debug build exists

Writing one element past the end of an array often passes the samples by luck. The release build hides the bug. The debug build stops with a message:

```
test   2  RE        29 ms
     Error: attempt to subscript container with out-of-bounds index 5,
     but container only holds 2 elements.
```

Writing a generator and a brute force

1. **Brute force:** the simplest correct solution, ignoring speed.
2. **`gen.py`:** print one valid, small input per run. The seed is passed as an argument, so every failure can be reproduced.
3. **Run stress.** On a mismatch, the input is saved to `stress_fail/input.txt`. Read it, fix `sol.cpp` or `brute.cpp`, and run again.

Small inputs are best: they keep the brute force fast and make bugs easy to see.

CONTEST DAY

The routine.

1. Start the listener and keep it running for the whole contest.
2. Click Competitive Companion on each problem. Open `sol.cpp`.
3. For tricky problems, write `brute.cpp` and `gen.py` first. They're cheap insurance.
4. Press `Space` `b` after each change. Fix every WA before moving on.
5. Before submitting: `Space` `d`, then `Space` `s` if you have a brute force.
6. Submit `sol.cpp`. The `dbg` lines do nothing on Codeforces.

TROUBLESHOOTING

When something is off.

Start with `python3 scripts/doctor.py`. It checks the compiler, the header, the port, the editors and the example, and prints the fix for each problem.

No GCC found

Apple's `g++` is clang, which has no `bits/stdc++.h`, so cf-kit uses Homebrew GCC. Run `brew install gcc`. On Linux, `sudo apt install g++`. To point at a specific compiler, set `CP_CXX=/path/to/g++-16`.

Clicking Competitive Companion does nothing

The listener must be running and must print `saved ...`. Open `http://127.0.0.1:27121/` in a browser. It should say the listener is running. If it doesn't, another program may hold port 27121.

Compiles are slow

Build the precompiled header with `python3 scripts/pch.py`. It must be rebuilt after a compiler upgrade.

Vim keys do nothing in VS Code

Open the repository root as the folder (not a subfolder), and check that the Vim extension is enabled.

My answer is right, but the judge says WA

The problem may accept several answers. The judge compares with one stored answer, so check the output by hand. Floating-point answers are compared with a tolerance of 10^{-6}.

SCOPE

What this does and doesn't do.

AREA	STATUS	NOTES
macOS, Apple Silicon, Homebrew GCC	Tested	The primary target. Every script was run on it.
Linux	Best effort	The code avoids macOS-only paths. Not yet run on a Linux machine.
Sublime Text	Config tested	The files are generated and validated. The editor UI itself has not been run by the project.
Hidden tests	Not available	Only the statement's samples are saved. Stress testing covers some of the gap.
Interactive problems	Not supported	The judge runs one program on a fixed input.
Debugger	Not configured	Use `dbg` and the sanitizer build.